

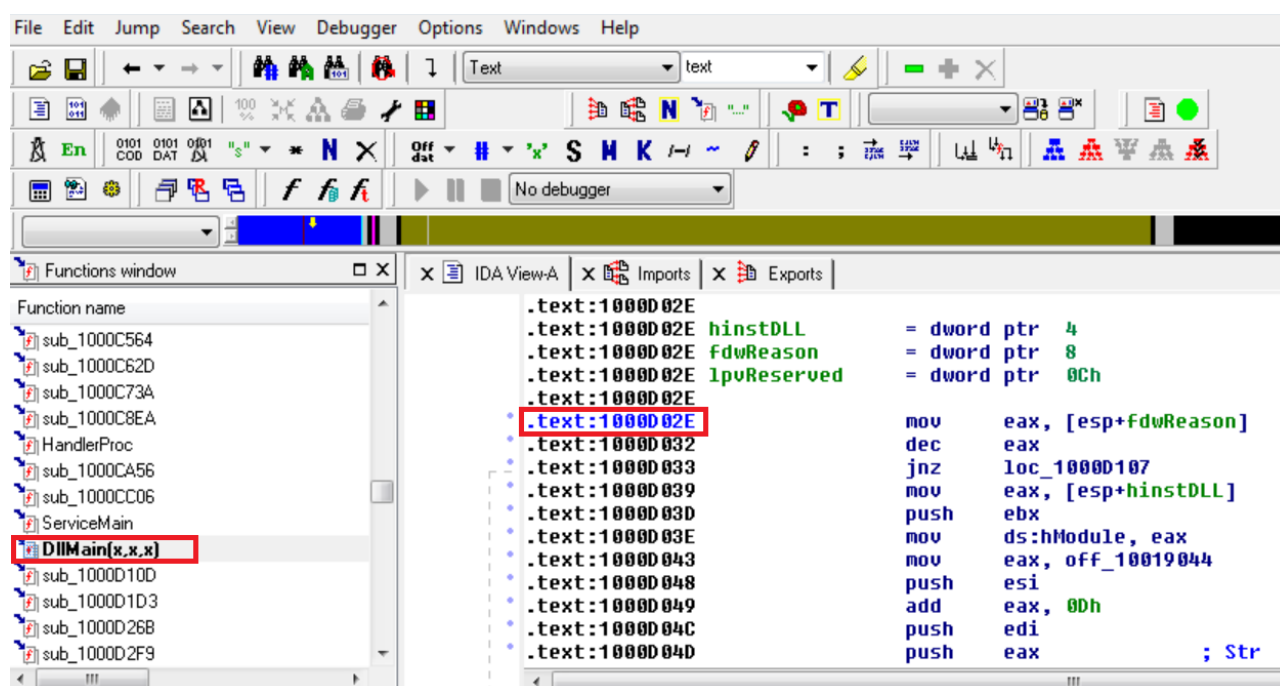
Analisi statica avanzata con IDA

Lo scopo dell'esercizio di oggi è di acquisire esperienza con IDA, un tool fondamentale per l'analisi statica. A tal proposito, con riferimento al malware chiamato «Malware_U3_W3_L2» presente all'interno della cartella «Esercizio_Pratico_U3_W3_L2» sul Desktop della macchina virtuale dedicata all'analisi dei malware, rispondere ai seguenti quesiti, utilizzando IDA Pro.

1. Individuare l'indirizzo della funzione DLLMain (così com'è, in esadecimale)
2. Dalla scheda «imports» individuare la funzione «gethostbyname». Qual è l'indirizzo dell'import? Cosa fa la funzione?
3. Quante sono le variabili locali della funzione alla locazione di memoria 0x10001656?
4. Quanti sono, invece, i parametri della funzione sopra?
5. Inserire altre considerazioni macro livello sul malware (comportamento)

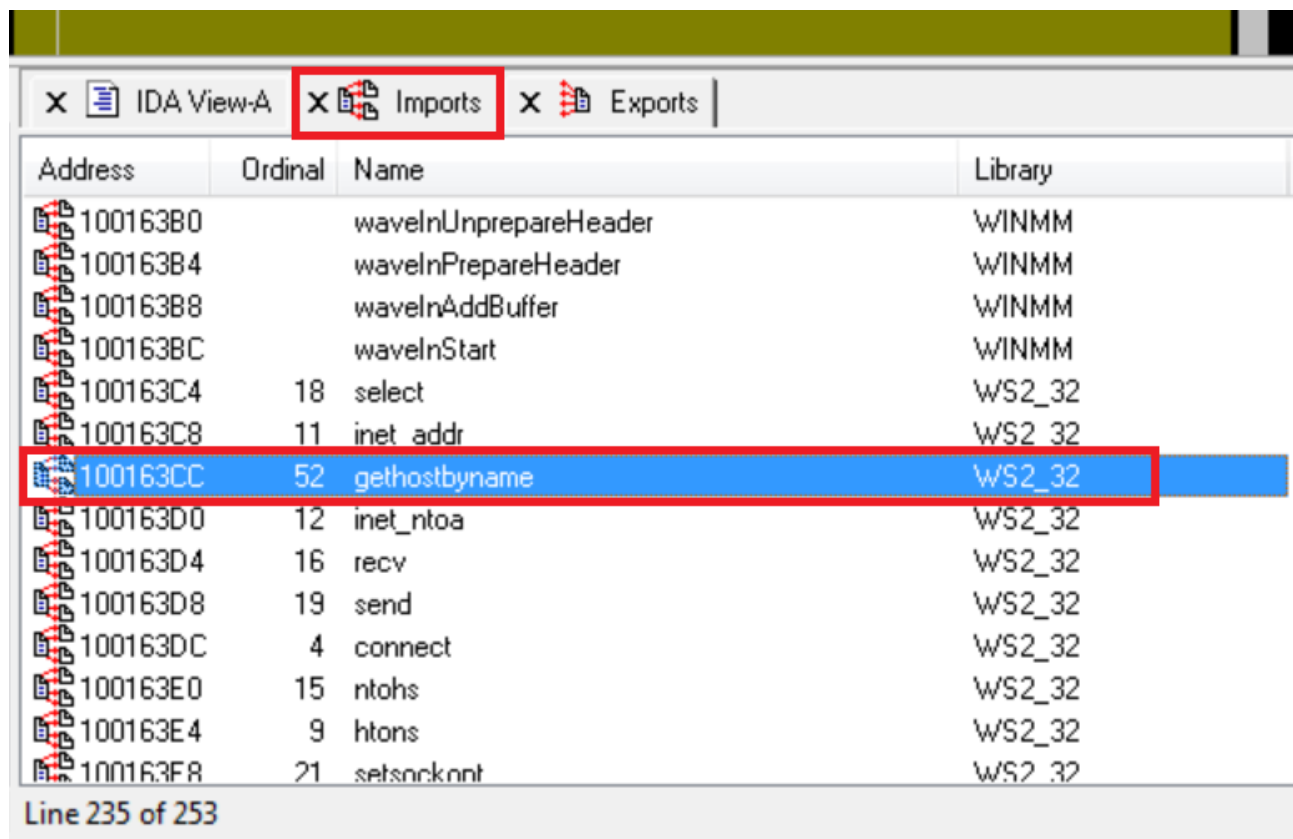
Individuare l'indirizzo della funzione DLLMain

Per vedere l'indirizzo della funzione DLLMain bisogna prima cercarla sulla barra delle finestre di funzione e cercare DLLNMain. A questo punto la schermata si aprirà automaticamente sul suo indirizzo di funzione.



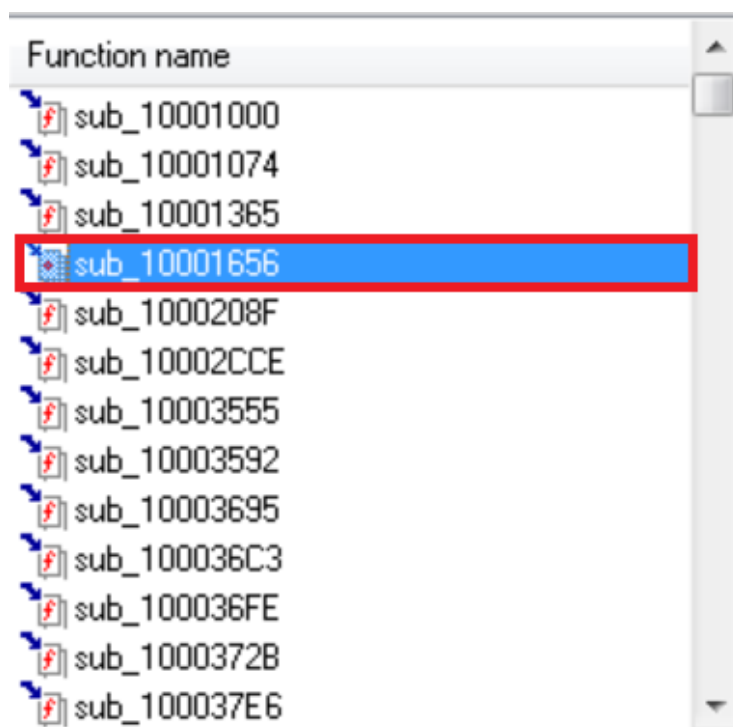
L'indirizzo della funzione è 100D02E

Dalla scheda «imports» individuare la funzione «gethostbyname». Qual è l'indirizzo dell'import? Cosa fa la funzione?



L'indirizzo dell'import è 100163CC. Cercando su google learn, la funzione «gethostbyname». La funzione gethostbyname recupera le informazioni host corrispondenti a un nome host da un database host

Quante sono le variabili locali della funzione alla locazione di memoria 0x10001656?



Per prima cosa bisogna spostarsi sulla funzione di memoria giusta usando la finestra sulla sinistra.

Dopodiché analizziamo la schermata che ci presenta IDA

```

; DWORD __stdcall sub_10001656(LPVOID)
sub_10001656 proc near

var_675= byte ptr -675h
var_674= dword ptr -674h
hLibModule= dword ptr -670h
timeout= timeeval ptr -66Ch
name= sockaddr ptr -664h
var_654= word ptr -654h
Dst= dword ptr -650h
Parameter= byte ptr -644h
var_640= byte ptr -640h
CommandLine= byte ptr -63Fh
Source= byte ptr -63Dh
Data= byte ptr -638h
var_637= byte ptr -637h
var_544= dword ptr -544h
var_50C= dword ptr -50Ch
var_508= dword ptr -508h
Buf2= byte ptr -4FCh
readfds= fd_set ptr -4BCh
phkResult= byte ptr -388h
var_380= dword ptr -380h
var_1A4= dword ptr -1A4h
var_194= dword ptr -194h
WSAData= WSAData ptr -190h
arg_0= dword ptr 4

sub     esp, 678h
push    ebx
push    ebp
push    esi
push    edi
call    sub_10001000
test    eax, eax
jnz     short loc_1000168C

```

Dalla schermata possiamo individuare diverse variabili.
Le variabili sono:

var_675 = byte ptr -675h

var_674 = dword ptr -674h

hLibModule = dword ptr -670h

timeout = timeeval ptr -66Ch

name = sockaddr ptr -664h

var_654 = word ptr -654h

Dst = dword ptr -650h

Parameter = byte ptr -644h

var_640 = byte ptr -640h

CommandLine = byte ptr -63Fh

Source = byte ptr -63Dh

Data = byte ptr -638h

var_637 = byte ptr -637h

var_544 = dword ptr -544h

var_50C = dword ptr -50Ch

var_508 = dword ptr -508h

Buf2 = byte ptr -4FCh

readfds = fd_set ptr -4BCh

phkResult = byte ptr -388h

var_380 = dword ptr -380h

var_194 = dword ptr -194h

WSAData = WSADATA ptr -190h

Per un totale di 22 variabili locali.

Quanti sono, invece, i parametri della funzione sopra?

I parametri della funzione sopra evidenziata è soltanto uno. Ed è arg_0 = dword ptr 4.

Inserire altre considerazioni macro livello sul malware

Cercherò di analizzare la prima parte della funzione 10001000

```
Dest= byte ptr -104h
var_103= byte ptr -103h

push    ebp
mov     ebp, esp
sub     esp, 104h

and     [ebp+Dest], 0
push    esi
push    edi
push    40h
pop     ecx
xor     eax, eax
lea     edi, [ebp+var_103]
push    104h                ; Count
rep stosd
stosd
stosb
lea     eax, [ebp+Dest]
push    eax                ; Dest
call    ds:GetCurrentProcessId
push    eax                ; int
call    sub_10003C00
mov     esi, ds:_stricmp
lea     eax, [ebp+Dest]
push    offset Str2        ; "rundll32.exe"
push    eax                ; Str1
call    esi ; _stricmp
add     esp, 14h
test    eax, eax
jz      short loc_10001060
```

Dichiarazione delle Variabili Locali:

Dest = byte ptr -104h

var_103 = byte ptr -103h

Dest: Questa variabile è un puntatore a un byte, situata a un offset di -0x104 dal registro EBP.

var_103: Un'altra variabile, anch'essa un puntatore a un byte, situata a un offset di -0x103 dal registro EBP.

Queste dichiarazioni ci informano che Dest e var_103 rappresentano spazi di memoria allocati sullo stack e sono utilizzati per operazioni successive all'interno della funzione.

Revisione della Funzione

Con queste dichiarazioni, il codice che hai fornito fa quanto segue:

Setup dello Stack Frame e Allocazione della Memoria:

```
push    ebp
mov     ebp, esp
sub     esp, 104h
```

Questo prepara lo stack frame, come detto in precedenza, e riserva 0x104 byte di spazio sullo stack per le variabili locali, in particolare per Dest e var_103.

Inizializzazione di Dest:

```
and     [ebp+Dest], 0
```

Qui Dest viene azzerato. Poiché Dest è a -0x104 da EBP, questa istruzione imposta il byte situato a [EBP-0x104] a 0.

Inizializzazione di var_103 e del Buffer:

```
lea     edi, [ebp+var_103]
push    104h          ; Count
rep stosd
stosw
stosb
```

Qui viene inizializzata un'area di memoria di 0x104 byte partendo da var_103, che è a -0x103 da EBP. Questo spazio viene riempito con 0x00 (grazie alla precedente xor eax, eax che azzerava eax).

Ottenimento dell'ID del Processo e Confronto con "rundll32.exe":

```
lea    eax, [ebp+Dest]
push   eax                ; Dest
call   ds:GetCurrentProcessId
push   eax                ; int
call   sub_10003C0D
mov     esi, ds:_stricmp
lea     eax, [ebp+Dest]
push   offset Str2        ; "rundll32.exe"
push   eax                ; Str1
call   esi ; _stricmp
```

L'ID del processo corrente viene confrontato con "rundll32.exe" per vedere se il processo attualmente in esecuzione è proprio rundll32.exe.